

GenAI Interview Preparation Guide

Java Developer & DevOps Engineer Edition

Difficulty Legend

Basic	Foundational concepts — expected in any GenAI screening round
Intermediate	Applied knowledge — RAG, embeddings, context management, tool use
Advanced	Architecture & system design — senior/staff-level interviews
DevOps + AI	DevOps-specific GenAI integration — Jenkins, Terraform, K8s, AIOps

Section 1 — Core GenAI Fundamentals

#	Level	Question & Answer
1	Basic	What is Generative AI and how does it differ from traditional ML? Generative AI creates new content (text, images, code) by learning patterns from training data, using models like LLMs (GPT, Claude). Traditional ML classifies or predicts from existing patterns. GenAI is probabilistic and creative; classical ML is deterministic and categorical.
2	Basic	What is a Large Language Model (LLM)? An LLM is a deep learning model trained on massive text corpora using transformer architecture. It predicts the next token in a sequence, enabling text generation, Q&A, code writing, and more. Examples: GPT-4, Claude, Llama.
3	Basic	What is prompt engineering and why does it matter for developers? Prompt engineering is the art of crafting inputs to an LLM to get accurate, reliable outputs. For Java/DevOps devs calling LLM APIs, well-structured prompts reduce hallucinations, improve JSON output quality, and directly impact the usability of AI features.
4	Basic	What are tokens in the context of LLMs? Tokens are the smallest units an LLM processes — roughly 3–4 characters or 0.75 words on average. Pricing, context window limits, and response speed are all token-based. This affects API cost estimation and max input/output sizing in your service.
5	Basic	Name two Java frameworks used for GenAI integration. 1) Spring AI — official Spring ecosystem support for OpenAI, Azure OpenAI, Anthropic, and vector store integrations. 2) LangChain4j — Java port of LangChain, offering chains, agents, RAG pipelines, and document loaders natively in Java.

Section 2 — Intermediate GenAI Concepts

#	Level	Question & Answer
6	Intermediate	Explain RAG (Retrieval-Augmented Generation) and how you'd build it in Java. RAG grounds an LLM response in your own data. Architecture: 1) Index documents — split into chunks, convert to embeddings, store in a vector DB. 2) At query time — embed the user query, similarity search in vector DB, inject top results as context into the LLM prompt. In Java: use LangChain4j's EmbeddingStore + OpenAiEmbeddingModel, or Spring AI's VectorStore abstraction with Pinecone/pgvector.
7	Intermediate	What is an embedding and how is it used in semantic search? An embedding is a high-dimensional float vector representing the semantic meaning of text. Similar texts have vectors close together in vector space. For semantic search: embed the user query, perform cosine similarity search against indexed document embeddings, and retrieve top-k results. Libraries: OpenAI text-embedding-3-small, HuggingFace sentence-transformers.
8	Intermediate	How would you handle LLM hallucinations in a production Java backend? Strategies: 1) RAG — ground responses in verified data. 2) Output validation — parse and validate structured JSON responses programmatically. 3) Confidence thresholds — fall back to human review. 4) Function calling — constrain the LLM to defined Java methods. 5) System prompts enforcing factual-only responses.
9	Intermediate	What is function calling / tool use in LLMs and how do you implement it in Java? Function calling lets an LLM invoke a predefined function instead of generating free text. You define functions with JSON schemas; the LLM returns a structured call when needed. In Java via Spring AI or LangChain4j: define @Tool annotated methods, register with the model client, and handle tool result injection back into the conversation loop.
10	Intermediate	What is the difference between fine-tuning and prompt engineering? Prompt engineering shapes model behaviour at inference time — no training needed, fast and cheap, but limited in depth. Fine-tuning re-trains model weights on domain-specific data — expensive, requires quality datasets, but produces deeper adaptation. For most enterprise Java use cases, RAG + prompt engineering beats fine-tuning in cost-to-value ratio.
11	Intermediate	How do you manage conversation history (context window) in a Java chatbot? LLMs are stateless — you must send history with every request. Strategies: 1) In-memory message list for short sessions. 2) Sliding window — keep last N messages within token limits. 3) Summarization — compress older turns periodically. 4) Redis/DB persistence for multi-session continuity. Spring AI's MessageChatMemoryAdvisor handles this with configurable strategies.

Section 3 — Advanced Architecture & System Design

#	Level	Question & Answer
12	Advanced	Design a GenAI-powered document Q&A system using Java and Spring AI. Architecture: 1) Ingestion pipeline — PDF/DOCX reader → text splitter (chunk 512, overlap 50) → OpenAI embedding model → store in pgvector via Spring AI VectorStore. 2) Query pipeline — embed user question → top-5 similarity search → construct prompt with context → stream response from GPT-4o. 3) Spring Boot REST endpoint, async via CompletableFuture, Redis for session memory, rate limiting with bucket4j.
13	Advanced	How would you build an AI agent with tool use in Java? An AI agent is an LLM in a ReAct loop: Thought → Action (tool call) → Observation → repeat. In LangChain4j: define @Tool methods (e.g. searchDatabase(), sendEmail()), register with AiServices. Key concerns: max iteration guards to prevent infinite loops, tool error handling, structured logging for observability, and fallback behaviour when tools fail.
14	Advanced	How do you secure GenAI API calls in a Java microservice? 1) Never expose API keys client-side — proxy all calls through your Java backend. 2) Store keys in Azure Key Vault / AWS Secrets Manager, injected via Spring @Value. 3) Input validation to prevent prompt injection attacks. 4) Rate limiting per user with bucket4j. 5) PII scrubbing before sending data to external LLM APIs. 6) Audit log every LLM interaction for compliance.
15	Advanced	Explain vector database internals — how does ANN similarity search work at scale? Vector DBs use Approximate Nearest Neighbor (ANN) algorithms like HNSW or IVF for fast search at scale. HNSW builds a multi-layer graph; queries traverse from coarse top layers to fine bottom layers. Cosine similarity (dot product of unit vectors) is the standard metric for text embeddings. ANN sacrifices exact accuracy for speed — recall@k metrics drive production tuning.

Section 4 — DevOps + AI Integration

#	Level	Question & Answer
16	DevOps + AI	How can GenAI be integrated into a Jenkins CI/CD pipeline? 1) AI-powered test failure analysis — send failed build logs to an LLM via API call in a Jenkinsfile post-failure step and receive root cause summaries in Slack. 2) PR description generation using Groovy + OpenAI API. 3) AI-driven code review gates flagging high-risk changes. 4) Auto-generating release notes from commit history before deployment. Secure the API key as a Jenkins Credential — never hardcode in Jenkinsfile.
17	DevOps + AI	How would you use GenAI to assist with Terraform / IaC automation?

#	Level	Question & Answer
		1) Generate Terraform modules from natural language descriptions — always human-reviewed before apply. 2) Explain existing .tf files for onboarding. 3) AI-driven drift detection commentary — send terraform plan output to LLM for a human-readable risk summary. 4) RAG chatbot over your IaC codebase — index .tf files in a vector DB and query via natural language.
18	DevOps + AI	What is AIOps and how does GenAI fit into it? AIOps applies AI/ML to IT operations — automating monitoring, alerting, anomaly detection, and incident response. GenAI adds a natural language layer: 1) LLMs summarize multi-source alert storms into a single incident narrative. 2) Conversational interfaces for querying metrics. 3) Auto-generated runbooks and postmortems from incident timelines. Tools: Dynatrace Davis AI, AWS DevOps Guru, and custom LLM integrations.
19	DevOps + AI	How would you build an AI-powered Kubernetes troubleshooting assistant? Architecture: 1) Pull kubectl describe pod, logs, and events via Kubernetes Java client. 2) Embed and index these into a vector store updated on a schedule. 3) User asks a natural language question → RAG retrieves relevant pod state + logs → LLM generates root cause + fix recommendation. 4) Expose as a Slack bot or internal web UI. Auth via Kubernetes RBAC; LLM never has raw cluster access.
20	DevOps + AI	How do you securely manage LLM API keys in a Dockerized / Kubernetes environment? 1) Store API keys as Kubernetes Secrets (base64 encoded, etcd encrypted at rest). 2) Mount as environment variables into the container spec — never bake into Docker image. 3) For production: use Azure Key Vault with CSI Driver or AWS Secrets Manager with IRSA — secrets injected at pod startup. 4) Rotate keys on a schedule via GitOps pipeline. 5) Restrict access with RBAC to only the required service account.
21	DevOps + AI	How can LLMs improve incident postmortem generation in a DevOps team? Feed the LLM: alert timeline, Slack/Teams thread, Jira ticket history, and relevant log excerpts. Prompt it to produce a structured postmortem covering timeline, root cause, impact, contributing factors, and action items. Automate with a Jenkins/GitHub Actions job triggered on incident-close, with output saved to Confluence via API. Always have an engineer review the draft — LLMs can miss subtle causal chains.